MagpieMark: A Reproducible Sim-to-Real Benchmarking Framework for Single and Multi-UAV Systems

Jonathan Distler

Sibley School of Mechanical and Aerospace Engineering, Cornell University

November 3, 2026

Abstract

Quantifying the performance of individual unmanned aerial vehicles (UAVs), as well as the control policies governing coordinated UAV swarms, is becoming increasingly important as UAVs are integrated into a growing range of real-world applications. Benchmarking has long been used across engineering and computer science to evaluate algorithmic performance, computational efficiency, and task execution. Although several UAV benchmarking frameworks exist, few adequately address complex flight maneuvers, dynamic environments, or the coordinated behavior of multiple UAVs operating as a swarm. This paper presents an open-source simulation framework and a low-cost, easily manufacturable experimental testbed designed to systematically evaluate both individual UAVs and multi-UAV systems. The proposed framework quantifies key performance metrics, including task completion time, trajectory-tracking error, and coordination accuracy, to enable standardized comparisons across different hardware configurations and control policies. Additionally, a set of baseline control policies is developed using primitive flight actions, including forward, backward, lateral, and vertical movements, to establish a reference for evaluating more sophisticated autonomous control strategies. By integrating simulation and physical experimentation, the framework provides a foundation for investigating the simulation-to-reality gap and assessing the robustness of UAV control policies under increasingly complex operating conditions. Ultimately, this work seeks to establish a reproducible and accessible benchmarking methodology that enables future researchers to compare UAV capabilities and swarm coordination strategies quantitatively against a common baseline, thereby supporting the systematic development and evaluation of increasingly capable autonomous aerial systems.

Introduction

The National Institute of Standards and Technology (NIST) [Nat25] developed the Aerial Drone Tests to evaluate unmanned aerial vehicle (UAV) capabilities and, more importantly, to test operator proficiency. The framework consists of five evaluation levels: basic proficiency, maneuvering trials, maneuvering with payload functionality, obstructed-environment maneuvering, and confined-environment maneuvering. These tests provide repeatable measures of flight performance with and without a payload, precision, and operational reliability, establishing a foundation for comparing aerial robotic systems under controlled conditions. Given this standardization, NIST test performance can be used to compare different operator abilities with the same UAV, or to compare different UAVs with the same operator.

However, while these tests standardize maneuverability and operational proficiency, they do not comprehensively address several capabilities increasingly important to modern autonomous UAV systems, including multi-agent coordination, dynamic obstacle avoidance, onboard computational efficiency, and autonomous decision-making; not all UAVs require a payload, meaning that a proportion of the NIST framework evaluates functionality that doesn't apply to every UAV. Research by Burri et al. [BNG+16] highlights the importance of evaluating autonomous perception and localization using the EuRoC Micro Aerial Vehicle dataset, which provides synchronized sensor measurements and ground-truth trajectories for benchmarking state-estimation algorithms. Similarly, Loquercio et al. [LKR+21] demonstrate the importance of onboard computation and real-time decision-making

through autonomous high-speed flight in complex environments, using policies trained in simulation and transferred to physical UAVs.

Beyond individual vehicle autonomy, the growing application of coordinated aerial systems introduces additional performance considerations. Preiss et al. [PHSA17] developed Crazyswarm, a platform that coordinates large groups of nano-quadcopters, highlighting the importance of evaluating multi-agent control, scalability, and coordinated flight behavior. Furthermore, Song et al. [SNK+21] introduced Flightmare, a flexible quadrotor simulation environment supporting reinforcement learning, obstacle avoidance, and parallel UAV simulation. Such environments enable researchers to develop and evaluate autonomous control policies across repeatable scenarios before physical deployment. Kaufmann et al. [KBL+23] further demonstrated the effectiveness of simulation-trained reinforcement learning policies by having autonomous UAVs race against human champions, illustrating both the potential of learned control policies and the importance of evaluating their transfer to real-world systems.

These studies show that autonomous aerial robotics research has moved beyond conventional flight-performance evaluation with static obstacles and manual controls toward sophisticated perception, computation, and autonomous control. Yet, benchmarking tests evaluate different UAV features and factors but lack a unified framework to generalize across autonomous policy effectiveness, computational latency, rich interactions in dynamic environments, swarm coordination, and simulation-to-reality results. A standardized benchmark incorporating all these factors would enable comparisons across autonomous UAV systems and provide a strong foundation for evaluating emerging features in both UAV and nascent swarm technologies.

UAV Architecture

To understand the motivations behind the simulation, one must understand the current state of UAVs for the Magpie suite developed by Mehlman et al. [MKL+25]. For a UAV to take off and fly with a given command, first it needs to estimate its position and orientation. The on-board flight controller uses sensors like the gyroscope and accelerometer to measure rotation and acceleration of the UAV. Then, the PX4 software combines these along with available positional measurements to estimate the position, velocity, and attitude of the UAV. To send a target, the UAV first needs to have a bound on its current position; the *startup* command captures the current position of the UAV. The *arm* command requests permission from PX4 to enable motors and enter an *Offboard mode*, where an external program provides the movement goals of the UAV. Then, *takeoff* keeps the current horizontal position and changes the altitude of the target from a stream of ground-height values when in *startup* to the desired height of the UAV. The propellers push air downward, creating a thrust force. When the upward thrust is greater than the UAV's weight ($Tcos(\theta) > mg$), the UAV rises.

Once a new position is commanded, the PX4 software provides the UAV with a desired position and desired velocity; then the PX4 software determines the necessary acceleration and thrust. The acceleration and thrust are mapped to a required thrust vector, which is then delegated to individual motor outputs. The motors are driven by electronic speed controllers (ESCs), with a closed-loop control in place in which the sensors provide feedback, letting PX4 check the result and adjust the necessary motor outputs.

In order to move forward, the UAV briefly changes the balance between the thrust of the motors, such that the UAV is pitched forward. Then, the thrust vector points partially forward, accelerating the UAV horizontally. Similarly, the total thrust is adjusted to support its weight while tilted. In order to brake, the tilt is produced in the opposite direction of velocity. In order to change its heading, the UAV changes the symmetry between clockwise-spinning and counterclockwise-spinning, resulting in a turning force. To hover, the upward thrust exactly balances the weight of the UAV. And, as will be touched on later, the *stop* command captures the current position of the UAV at the time of the command as a hold target.

Simulation Methodology

The simulation aimed to mimic the real-world setup and control of UAV systems. This involved defining the aforementioned primitives for each action of the UAV sequence, including *startup*, *arm*,

and *takeoff*, along with primitives for movement such as *forward*, *backward*, *right*, *left*, *up*, and *down*. All movements are relative to the UAV's position in the global reference frame, intended to closely match the motion sensing in our hardware testbed.

Single UAV Simulation Environment Setup

The UAV environment was built using the high-level goals of the NIST-defined tests. Single-UAV functionality was added to address the dynamic environments found in many real-world systems, as shown in figure 1. The tests were intended to be conducted from left to right. First, this included an analog to the NIST test one (basic proficiency), which involved weaving through a set of cones; the CAD representation included ArUco tags that can be used in future tests for providing computational metrics and computer-vision compatibility. Second, the flag was added to address the UAV's dynamic capabilities. The flag has a mechanism to turn either left or right, and the UAV would respond and fly around the flag in the direction in which the flag was pointing; for example, if the flag pointed to the right, the UAV would use an on-board mechanism to detect the change in orientation and fly around the flag in the right direction. Finally, the confined space flight was added in direct reference to tests four and five outlined by NIST in their confinement maneuvering tests, but was redesigned to match the form factor of the other tests.

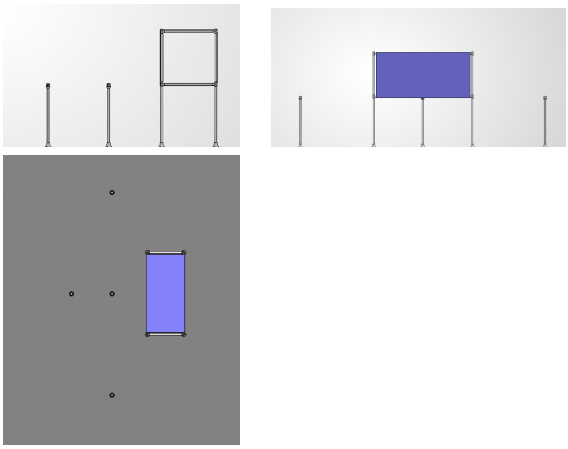


Figure 1: UAV Benchmark CAD Representations

After developing the computer-aided design (CAD) representations, the UAV simulation environment was configured using Gazebo as the physics engine, PX4 as the flight-control and autopilot software, and MAVProxy as the MAVLink-based communication interface. The individual CAD assemblies were then imported into a Gazebo simulation world alongside an X500 quadrotor, enabling evaluation of UAV behavior and interactions in a controlled virtual environment.

To promote accessibility, reproducibility, and further development, the simulation environment was built using open-source software. The development environment included Ubuntu 24.04, Python 3.12, ROS 2 Jazzy, and Gazebo Harmonic, along with PX4 and MAVProxy. The project's open-source GitHub repository, UAV Benchmark, includes the relevant simulation files, dependencies, and installation instructions. To reproduce the simulation environment, users should download and extract $\texttt{Magpie}_{M}ark.zip.Usersshouldthenfollowtheinstructionsprovidedin$

Swarm UAV Simulation Environment Setup

Although the NIST aerial UAV benchmarking framework does not explicitly address multi-UAV swarm coordination, Preiss et al.'s work provides a foundation for evaluating coordinated flight and swarm behavior. Building on this research, three distinct benchmarking tests were added across two swarm system architectures to evaluate the proposed framework's generalizability. These tests assessed formation control, spatial reorganization, decentralized decision-making, and collective responses to dynamic commands. The project does not include an adjacent simulation environment, as the work was mainly interested in determining the quality of swarms, rather than their reaction to obstacles, which serves as an implementation for future work. The available testing space was also constrained, so introducing

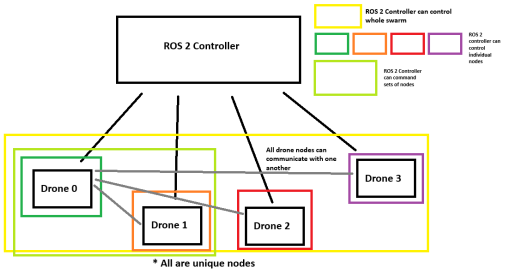


Figure 2: ROS 2 Central Command Controller

hard constraints would be infeasible, which provides direction for further work. All of the swarm tests outlined represent a swarm size of three UAVs; however, the simulation is set up to be generalizable to n unique UAVs; the research involved three UAVs as that was the limit on the number of working UAVs of the same build in the *Cornell Aerospace Adversary Lab* at the time of development.

The first test evaluates a UAV swarm's ability to maintain and reconfigure its formation under prescribed conditions. Initially, the UAVs are positioned in a linear formation along the y-axis. At the start of the test, the swarm is commanded to reorganize into a linear formation along the x-axis. The transition accuracy is quantified using the root mean-squared error (RMS) between the observed UAV positions and their corresponding target positions at discrete time intervals. This metric characterizes the swarm's formation-tracking accuracy throughout the reorientation maneuver.

The second test evaluates the swarm's ability to establish an organized formation from randomized initial conditions. Each UAV is initially assigned a randomly generated position within a bounded three-dimensional region, with a fixed random seed used to ensure reproducibility across repeated trials. Upon initiation, the UAVs are commanded to reorganize into a linear formation along the y-axis. The resulting formation error and convergence time quantitatively measure the swarm's ability to coordinate and recover an organized spatial configuration from initially disordered positions.

The third test extends the benchmarking framework to evaluate dynamic swarm partitioning and subsequent reunification, capabilities not explicitly addressed in the NIST benchmarking framework. Initially, the UAVs form a linear formation along the y-axis. At initiation, the swarm receives a directional flag, instructing a majority subset of UAVs ($\lfloor \frac{n}{2} \rfloor + 1$) to move in the indicated direction while the remaining UAVs move in the opposite direction. Following this separation, the two subsets are commanded to reconverge into a unified formation. Performance is assessed by the accuracy of the initial swarm partition, each subset's ability to execute its assigned trajectory, and the time required for the swarm to reunify successfully.

Collectively, these three tests extend conventional single-UAV benchmarking methods to multi-agent systems by introducing quantitative measures of formation control, spatial convergence, coordinated decision-making, and dynamic swarm reconfiguration. Evaluating these behaviors across two distinct system architectures provides a basis for examining the robustness and generalizability of the proposed benchmarking framework.

Swarm Architectures

The two system architectures are described as follows. The first architecture follows a central command hierarchy in which all UAVs subscribe to each other's nodes, and a centralized controller external to the system runs all autonomous actions, as shown in Figure 2. The second architecture is an edge UAV command hierarchy in which an arbitrary UAV serves as the controller, issuing all commands while also following them, as shown in Figure 3. The high-level difference is that the central command hierarchy involves an external agent, whereas the edge command involves an agent internal to the swarm.

The simulation requirements are the same as in *Single UAV Simulation Environment*, with all of the required scripts kept in the same `.zip`, under the same open-source GitHub.

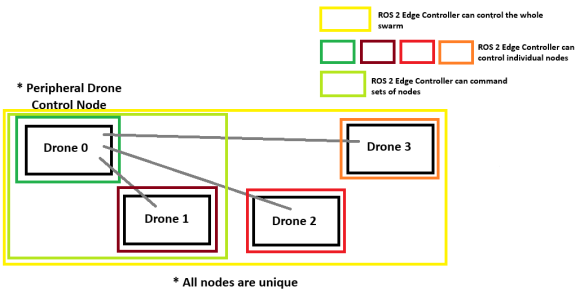


Figure 3: ROS 2 Edge Command Controller

Centralized Controller Command Benchmarks

For a centralized controller, the environment is first sourced in the primary terminal. Then, one should follow the commands below, with the f-string integer following n representing the intended number of UAVs in the swarm. As an aside, like the single UAV benchmark tests, all results are saved to a *.csv* file. As an aside, all UAVs are assigned an index, starting at 0 through $n-1$

UAV Swarm Test Central Controller - Console Setup

```
/usr/bin/python3 scripts/start_simulation.py --world swarm --n 3
```

Then, in a second terminal, one should again source the environment, and then run the same sequence of commands, including `startup`, `arm`, and `takeoff`, as in the single UAV benchmarks. It is paramount to keep the same randomization seed so that the tests are generalizable under any policy:

UAV Swarm Test Central Controller - Console Commands

```
ros2 run swarm_mavsdk_ros2 swarm_tests --n 3 --seed 42

startup
arm
takeoff

test 1
test 2
test 3
land
quit
```

Edge Controller Command Benchmarks

For an edge controller, again one should source the environment in the first terminal. Then, following the commands below, with the integer following n representing the number of UAVs in the swarm, and the integer following *architecture edge: drone* being the ID of the controlling UAV:

UAV Swarm Test Edge Controller - Console Setup

```
python scripts/start_simulation.py
{ros conda --world swarm --n 3 --architecture edge:drone_0
```

Then, in the second terminal, source the environment again and run the following commands. Again, make sure to keep the same seed across tests for generalizability:

UAV Swarm Test Edge Controller - Console Commands

```
swarm() {python -m swarm_mavsdk_ros2.swarm_command \ --n 3 --architecture edge:drone_0 \$@"}

swarm startup
swarm arm
swarm takeoff

swarm test 1 --seed 42
swarm test 2 --seed 42
swarm test 3 --seed 42

swarm land
swarm disarm
```

Simulation-to-Real Transfer

The proposed benchmark employs a unified ROS 2 and PX4 architecture to maintain consistent control and communication interfaces across simulated and physical environments. At the highest level, ROS 2 coordinates vehicle behavior by issuing commands such as takeoff and transmitting position setpoints to individual UAVs. Each vehicle is assigned a logical identifier, such as *drone 0* or *drone 1*, which is maintained across both environments and displayed through the monitoring dashboard. This abstraction allows the research controller to address individual vehicles independently of the underlying simulation or hardware configuration, reducing the modifications required when transitioning between experimental environments.

PX4 provides the flight-control functionality in both environments, although its execution differs between simulation and hardware. In the simulated environment, PX4 operates in software-in-the-loop (SITL) mode, receiving simulated sensor measurements and computing the control outputs required to govern the vehicle's motion within Gazebo. In the physical environment, PX4 executes on the onboard flight controller, processes measurements from the UAV's sensors, and generates control signals for the ESCs, which regulate the individual motors. Consequently, both environments employ the same underlying flight-control framework, providing continuity in the control architecture while preserving the distinct characteristics of simulated and physical flight. However, compatibility between PX4 firmware versions, ROS 2 message definitions, and communication interfaces remains a necessary prerequisite to ensure consistent operation.

The primary distinction between the two environments arises from the acquisition of vehicle state measurements. Within Gazebo, the physics engine directly computes each vehicle's position and orientation. A simulated tracking interface extracts these measurements and publishes them as standardized ROS 2 odometry messages, reproducing the interface of a typical external localization system. In the physical laboratory, vehicle localization is performed using an external tracking system, initially implemented with Marvelmind and subsequently replaced by Vicon motion capture. The motion-capture system measures the position and orientation of rigid bodies associated with individual UAVs, and a corresponding ROS 2 driver publishes these measurements through the same standardized interface. Although the simulation does not explicitly reproduce the sensing physics, measurement uncertainty, or processing characteristics of the physical motion-capture system, it provides an equivalent external-odometry interface. This allows the downstream estimation and control architecture to remain consistent between the two environments.

An external-odometry adapter subsequently transforms the tracking measurements into the coordinate conventions and message formats required by PX4. This transformation is necessary because external tracking systems and flight controllers may represent position and orientation using different reference frames. For example, PX4 commonly employs a North-East-Down (NED) inertial reference frame and a Forward-Right-Down (FRD) body frame, whereas external tracking systems may use alternative coordinate conventions. The adapter applies the appropriate transformations to position and orientation measurements while accounting for the spatial offset between the tracked rigid body and the UAV's designated body-frame origin. Such corrections are particularly important when tracking

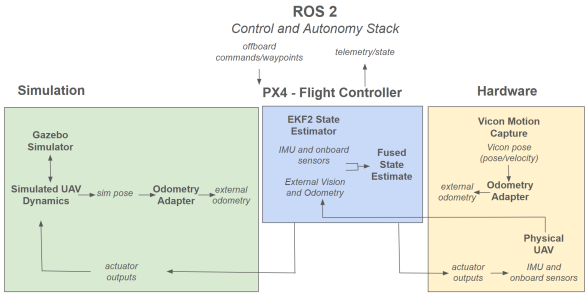


Figure 4: Simulated versus Real PX4 Interfaces

markers are displaced from the vehicle's reference point, such as any offset from the centroid of the UAV. The adapter also maintains measurement timestamps, accounts for the timing requirements of the receiving system, and rejects invalid measurements before publishing the resulting odometry data to PX4.

The transformed external-odometry measurements are incorporated into PX4's Extended Kalman Filter (EKF2), which combines external localization information with onboard inertial measurements to estimate the vehicle's state. This sensor-fusion process produces state estimates including position, velocity, and attitude, which are subsequently used for feedback control. In the proposed simulation architecture, the external tracking interface supplies position and orientation measurements without independently measured linear velocity. The velocity must be estimated through the state-estimation process rather than directly obtained from the external tracking interface. Keeping this distinction is necessary because it ensures that the simulation does not provide the controller with additional externally measured state information that would be unavailable through the corresponding physical tracking system.

The research controller closes the feedback loop by continuously generating desired vehicle setpoints based on the estimated state and the prescribed experimental objectives. For example, when a desired position is specified, the corresponding ROS 2 vehicle node transmits the position setpoint and Offboard-control heartbeat messages to PX4 at a frequency of 20 Hz. PX4 then uses its estimated vehicle state to calculate the control actions necessary to reduce the deviation between the desired and actual positions. In simulation, these control outputs are applied to the vehicle model within Gazebo, where the physics engine determines the resulting motion. The updated simulated state is subsequently obtained through the tracking interface, transformed by the odometry adapter, and incorporated into the PX4 estimator. In the physical environment, the same process is performed using measurements from the motion-capture system, while the onboard flight controller actuates the physical motors to produce the required vehicle motion. In both cases, successive state measurements update the feedback available to the controller, allowing it to continuously adjust the commanded trajectory in response to the vehicle's observed behavior.

By preserving the control, estimation, and communication interfaces across both environments, the MagpieMark benchmark provides a consistent framework for evaluating the transferability of autonomous UAV behaviors from simulation to physical experimentation. Nevertheless, equivalent software interfaces do not guarantee identical vehicle responses. Differences in vehicle mass properties, aerodynamic effects, actuator dynamics, controller tuning, sensor uncertainty, communication latency, and external disturbances may produce discrepancies between simulated and experimental trajectories. Consequently, the benchmark evaluates simulation-to-hardware transfer using quantitative performance metrics, including trajectory-tracking error, settling time, vehicle separation, and task-completion rate. These measures provide a basis for characterizing discrepancies between the two environments and assessing the extent to which control strategies developed in simulation retain their effectiveness during physical operation. A visual breakdown of the differences between simulation and real hardware is shown in Figure 4

Results

Single UAV Tests

The primary objective of the single-UAV simulations was to evaluate tracking accuracy by comparing the commanded and measured positions over time. Performance was assessed using the instantaneous position error and the root-mean-square error (RMSE), providing measures of overall tracking accuracy. The instantaneous error at each time step was defined as the Euclidean distance between the commanded position, $\mathbf{p}\text{target}, i$, and the measured position, $\mathbf{p}\text{actual}, i$:

$$e_i = |\mathbf{p}\text{target}, i - \mathbf{p}\text{actual}, i|\, 2, \ \text{RMSE} \qquad = \sqrt{\frac{1}{N} \sum i = 1^N e_i^2}, \tag{1}$$

where N represents the total number of recorded samples. The position error was plotted versus elapsed time to evaluate convergence and overall tracking performance. Similarly, the commanded and measured trajectories were plotted in the horizontal plane to visualize the waypoint-following behavior. Figures 5, 6, and 7 present the results of the three simulation tests.

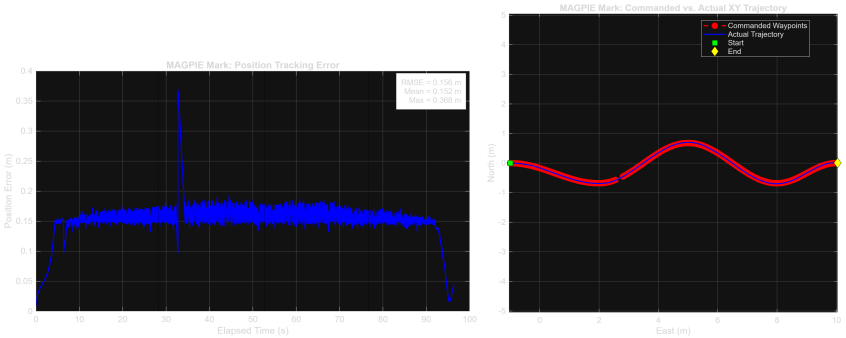


Figure 5: MagpieMark Single UAV Test 1: Position tracking error over time and commanded versus actual XY trajectory. The position error RMSE was 0.1557 m; mean position error was 0.1517 m; maximum position error was 0.3679 m.

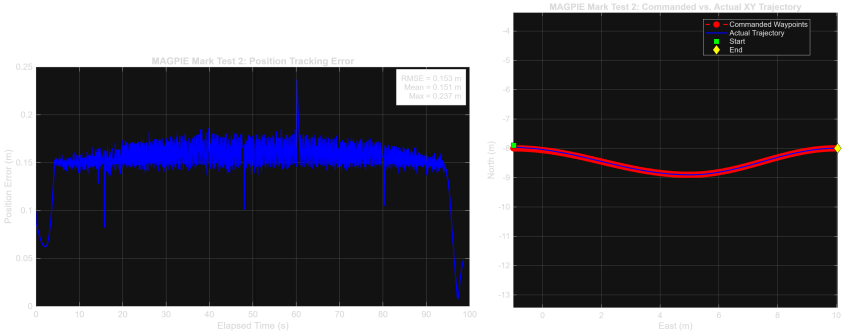


Figure 6: MagpieMark Single UAV Test 2: Position tracking error over time and commanded versus actual XY trajectory. The position RMSE was 0.1531 m; mean position error was 0.1505 m; maximum position error was 0.2369 m.

As illustrated in figures 5 and 6, the UAV demonstrates relatively accurate waypoint tracking throughout the first two tests, with generally small positional deviations interrupted by an occasional transient increases in error. This is corroborated by the position error RMSE and mean position errors. These results suggest that the control strategy is capable of maintaining satisfactory tracking performance under the corresponding test conditions. In contrast, Figure 7 exhibits substantially larger positional deviations, indicating reduced tracking accuracy and a higher overall RMSE. This discrepancy suggests that the existing control strategy may not generalize equally well across the

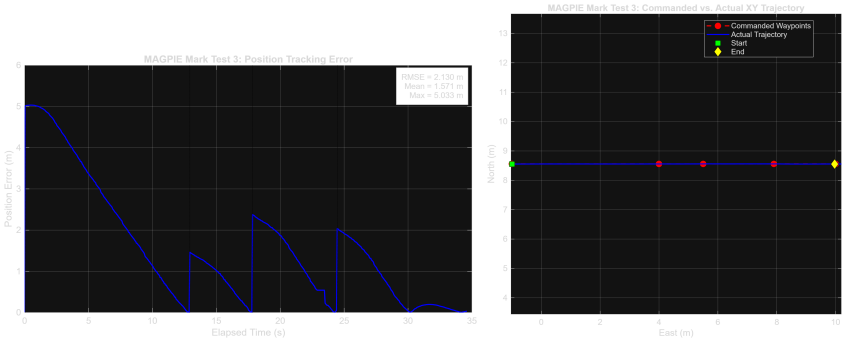


Figure 7: MagpieMark Single UAV Test 3: Position tracking error over time and commanded versus actual XY trajectory. The position RMSE was 2.1304 m; mean position error was 1.5709 m; maximum position error was 5.0331 m.

tested trajectories or operating conditions. Further investigation into controller tuning, commanded velocities, and waypoint transitions may help identify the sources of the increased tracking error and improve performance consistency.

Swarm Tests

Multi-UAV Swarm Tests

The primary objective of the multi-UAV swarm simulations was to evaluate the ability of multiple vehicles to execute coordinated maneuvers while maintaining accurate waypoint tracking and a viable spatial separation. Unlike the single UAV test suite, which focused primarily on individual tracking accuracy, the swarm experiments examined the collective behavior of three UAVs operating simultaneously. The commanded waypoints and measured trajectories were plotted in the horizontal plane to visualize the navigation behavior of each vehicle. Additionally, the minimum UAV separation within the swarm was evaluated as a function of time, providing a measure of the closest distance between any two vehicles throughout each experiment. This metric is particularly relevant for assessing vehicle proximity and identifying potential collision risks during coordinated maneuvers, especially in adversarial environments.

Figures 8, 9, and 10 present the results of the three swarm simulation tests, including the minimum inter-UAV separation and the commanded versus measured trajectories.

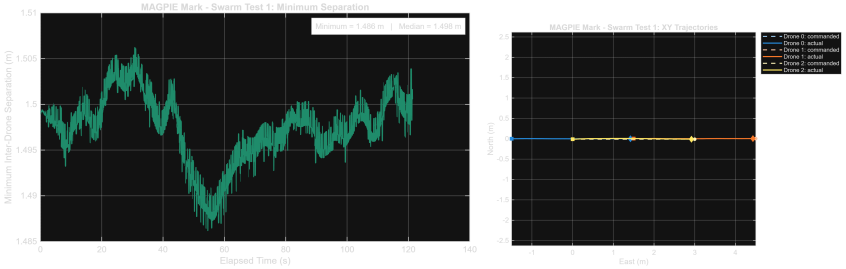


Figure 8: MAGPIE Mark Swarm Test 1: Minimum inter-UAV separation over time (left) and commanded waypoints versus measured XY trajectories (right).

The results demonstrate varying levels of tracking accuracy across the three swarm experiments. In Test 1, the individual position-error RMS values indicated that all three vehicles experienced comparable tracking errors. In contrast, Test 2 exhibited substantially greater variation between vehicles. Although drone 0 maintained relatively accurate waypoint tracking, drones 1 and 2 experienced larger deviations, indicating nonuniform tracking performance across the swarm.

Test 3 demonstrated more consistent tracking accuracy among the three vehicles. Compared with Test 1, these values indicate a substantial reduction in tracking error across all three UAVs. Fur-

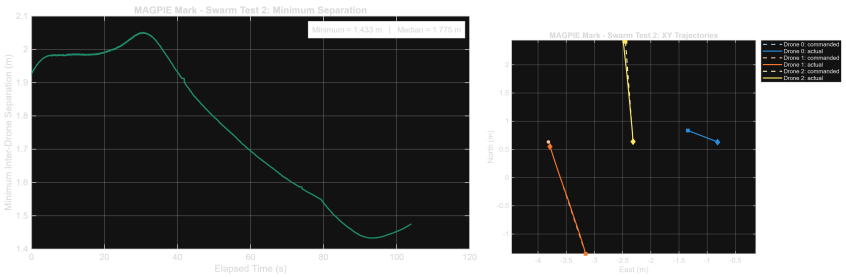


Figure 9: MAGPIE Mark Swarm Test 2: Minimum inter-UAV separation over time (left) and commanded waypoints versus measured XY trajectories (right).

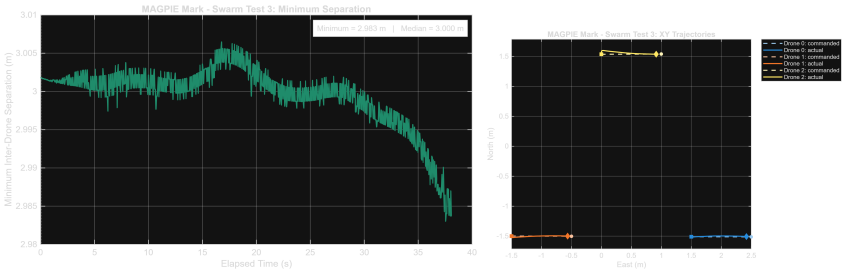


Figure 10: MAGPIE Mark Swarm Test 3: Minimum inter-UAV separation over time (left) and commanded waypoints versus measured XY trajectories (right).

thermore, the small variation between the individual RMS errors suggests more uniform tracking performance during this experiment.

Collectively, these results demonstrate the importance of evaluating both individual vehicle performance and unique behavior among swarms when assessing multi-UAV control strategies. While individual RMS errors characterize waypoint-tracking accuracy, minimum separation provides an additional measure of swarm proximity during coordinated maneuvers. Evaluating these metrics together allows for a more comprehensive assessment of swarm performance and provides a basis for identifying conditions under which further controller tuning or coordination improvements may be necessary.

Table 1: RMS Position Error Across Tests

Test	**Drone 0**	**Drone 1**	**Drone 2**
Test 1	1.7882	1.7483	1.7851
Test 2	0.1977	1.2681	1.0318
Test 3	0.6344	0.6335	0.6395

All values are in meters (m).

Localization

Previous Localization Setup

Previous work focused on developing an indoor localization framework for a single UAV using the Marvelmind Indoor Positioning System [Mar26]. Under favorable operating conditions, the system demonstrated positional accuracy approaching the manufacturer's reported ± 2 *cm*. However, despite its relatively high positional accuracy, several limitations emerged that hindered its suitability for more complex autonomous flight operations and multi-UAV benchmarking.

One significant limitation was the system's inability to independently determine the UAV's full six-degree-of-freedom (6-DoF) pose. While the localization system provided translational position estimates, it did not directly measure the vehicle's roll, pitch, or yaw, which required additional modems, necessitating the integration of additional onboard sensors or state-estimation techniques. Furthermore, the system exhibited high latency. Anecdotally, these delays occasionally became sufficiently

pronounced to compromise the responsiveness and stability of autonomous flight. As a consequence, the achievable localization accuracy did not translate into equally precise trajectory tracking or autonomous control.

Additional challenges emerged when considering the system's scalability. Although the Marvelmind platform offered a relatively accessible alternative to more expensive indoor motion-capture systems, the upfront infrastructure requirements and additional hardware needed to expand the system created practical and financial barriers to multi-UAV experimentation. Moreover, increasing the number of independently tracked vehicles would further strain communication, state estimation, and computational resources. These considerations made the architecture less suitable for developing a generalized benchmarking framework that could accommodate multiple UAVs operating simultaneously.

The latency and architectural constraints also complicated the development of a real-time digital twin. Ideally, a digital twin should continuously incorporate measurements from the physical UAV and maintain a sufficiently synchronized representation of its motion within the simulation environment. However, localization delays prevented a bijective correspondence with the simulation.

These limitations informed the design objectives of the present benchmarking framework, which we outline in **Current Motion Sensing Setup**. Despite these limitations, the Marvelmind system remains a viable option for applications that prioritize low-cost indoor positional localization over high-frequency feedback control and full-pose estimation. We made the implementation, configuration, and ROS 2 integration developed in the preceding work publicly available at Marvelmind Localization Methods via ROS 2, providing a reference for future researchers interested in using similar indoor localization methods.

Current Motion Sensing Setup

As previously discussed, the Marvelmind localization system presented several limitations, including poor scalability to multiple UAVs, an inability to directly measure UAV attitude, and latency that limited its suitability for real-time digital twin applications. To address these challenges, the laboratory invested in a Vicon 3D motion capture system.

The Vicon system uses multiple motion-capture cameras positioned around the testing environment to track optical markers attached to each UAV. Unlike the Marvelmind system, whose effective coverage and scalability constrained previous experiments, Vicon provides high-precision position and orientation measurements within a calibrated capture volume. These measurements can estimate UAV velocity and track multiple vehicles simultaneously, enabling high-fidelity instrumentation of swarm behavior. Although the system has not yet been fully implemented, it is expected to provide substantially lower latency and greater localization accuracy than the previous Marvelmind setup. However, we must establish the extent of these improvements experimentally. Ideally, the system will enable real-time comparisons between physical UAV trajectories and their simulated counterparts, supporting digital twin development and providing quantitative metrics for evaluating control policies.

Despite these advantages, the Vicon motion capture system is prohibitively expensive for many research environments, limiting its accessibility and scalability beyond dedicated laboratory facilities. Future work should therefore investigate more affordable localization alternatives that achieve lower latency and higher accuracy than the Marvelmind system. Nevertheless, reliable external localization remains essential to experimentally validating the MagpieMark benchmark. Without sufficiently accurate measurements of UAV position and orientation, simulated trajectories cannot be meaningfully compared against physical flight behavior, limiting the ability to quantify tracking error, swarm coordination, and the simulation-to-reality gap. Consequently, establishing a reliable ground-truth localization framework is critical to validating MagpieMark and enabling reproducible comparisons of UAV control policies.

Physical Test-System

As noted previously, the UAV requires a starting orientation and position in order to start up. In simulation, the *Gazebo x500* vision drone is used, which provides a stream of visual data to locate the UAV in simulation. The physical benchmark environment aims to provide a real-world analog to the simulated UAV benchmark. At a minimum, the physical environment must provide reliable localization measurements, including the UAV's position within a global reference frame, translational

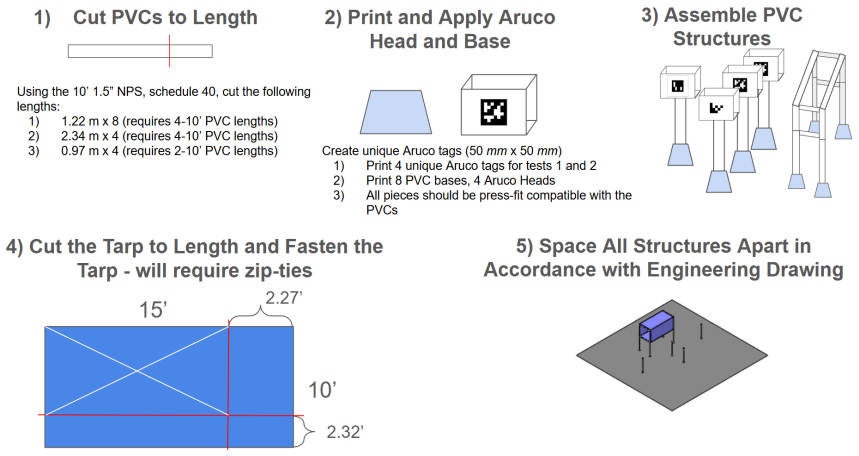


Figure 11: Physical Benchmark Assembly

velocity, and attitude (roll, pitch, and yaw). These measurements are necessary to evaluate UAV performance quantitatively and establish a consistent basis for comparison between simulated and physical experiments.

Ideally, a control policy would first be trained in simulation to navigate the obstacle course before being transferred to the UAV's onboard computer for testing in the physical benchmark environment. The simulation metrics would serve as an idealized baseline against which experimental results could be compared. Differences in trajectory tracking, obstacle avoidance, flight stability, and overall task performance could then be quantified to evaluate the simulation-to-reality (sim-to-real) gap. This approach would enable systematic evaluation of policy robustness and identify limitations associated with transferring control strategies from simulation to physical hardware.

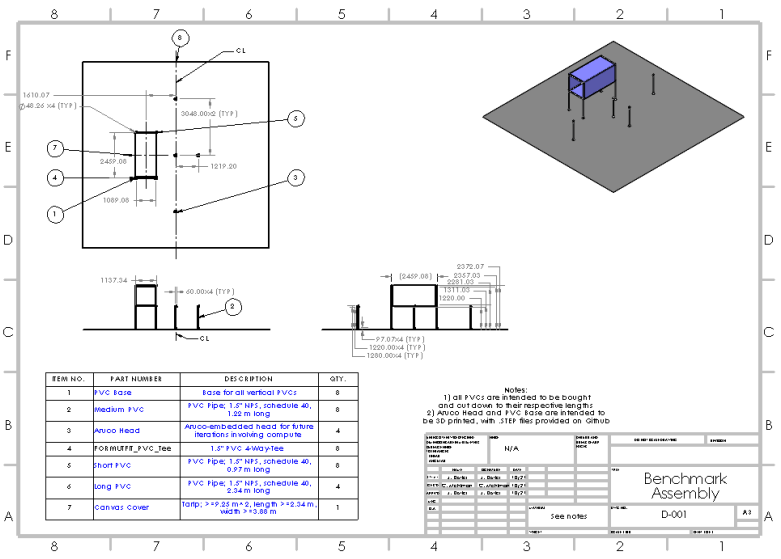
Additionally, the UAV configuration within an obstacle-free simulation environment, as documented in the accompanying GitHub repository, provides a generalizable framework for developing and evaluating control policies beyond the predefined benchmark course. Such policies could subsequently be transferred to physical UAVs for qualitative and quantitative evaluation. Nevertheless, reliable localization remains a fundamental requirement for meaningful physical benchmarking. At a minimum, the localization system must provide sufficiently accurate, timely state estimates that can be communicated via MAVLink-compatible messages, enabling the onboard flight controller to operate with a consistent representation of the UAV's position and motion.

UAV Physical Benchmark Environment Setup

The physical benchmark environment focuses on evaluating single UAV performance. MagpieMark also contains benchmarking for UAV swarms; these swarm benchmarks currently do not require physical obstacles. Future versions of MagpieMark could extend swarm benchmarks to include obstacle-rich environments, allowing assessment of swarm coordination, collision avoidance, and navigation in more complex scenarios. The current physical benchmark environment aims to replicate as closely as possible the single UAV simulation benchmarks. The physical benchmark environment as proposed can be constructed as per the diagram in Figure 11.

Bill of Materials and Test Setup Assembly

The original design of the experimental testbed is illustrated in figure 12. Initially, a separate testbed was proposed to evaluate multi-UAV swarm behaviors. However, because the proposed swarm benchmarking tasks were spatially translatable and could be performed without a fixed arrangement of physical obstacles, a dedicated obstacle course was deemed unnecessary, as this would also increase the cost of setup. The swarm experiments could be conducted in an open testing area with an appropriately defined spatial configuration. So, the experimental framework was simplified to accommodate both individual UAV and swarm-level benchmarking without requiring the construction of two distinct



ITEM NO.	PART NUMBER	DESCRIPTION	QTY.
1	PVC Base	Base for all vertical PVCs	8
2	Medium PVC	PVC Pipe, 1.5" NPS, schedule 40, 1.22 m long	8
3	Aruco Head	Aruco-embedded head for future iterations involving compute	4
4	FORMUFFT_PVC_Tee	1.5" PVC 4-Way-Tee	8
5	Short PVC	PVC Pipe, 1.5" NPS, schedule 40, 0.97 m long	8
6	Long PVC	PVC Pipe, 1.5" NPS, schedule 40, 2.34 m long	4
7	Canvas Cover	Tarp; > +9.25 m^2, length > +2.34 m, width > +3.66 m	1

Figure 12: Physical Benchmark Environment Setup

physical testbeds. The bill of materials is outlined at the following link: UAV Bill of Materials online reference, with the embedded hyperlinks providing the particular vendor from whom I purchased the materials.

Next Steps

Several opportunities remain for future development. First, incorporating ArUco markers and onboard computer vision could enable more sophisticated perception and localization capabilities, although these were beyond the scope of the present project. Second, introducing dynamic obstacles that change position over time would create a more challenging testing environment, requiring more adaptive, robust navigation policies. Similarly, extending the swarm benchmarks to include physical obstacles would allow for a more comprehensive evaluation of multi-UAV coordination, collision avoidance, and communication efficiency in complex environments. Future work could investigate alternative localization systems that offer improved accuracy, reduced latency, and lower implementation costs compared with the previously discussed Marvelmind and Vicon systems. Collectively, these developments would further establish MagpieMark as a reproducible and extensible framework for evaluating UAV control policies, onboard computation, and swarm coordination in both simulated and physical environments.

The current simulation uses the x500 quadcopter platform. Future iterations could investigate alternative UAV morphologies, such as *omnicopters* and *tricopters*, which would require different control policies to accommodate their distinct flight dynamics and maneuvering capabilities.

To better represent adversarial environments, future work could make use of adversarial agents capable of disrupting otherwise successful missions or introducing dynamic operational constraints. These scenarios could include GPS spoofing, communication jamming, and control-based attacks, requiring UAVs to adapt their control strategies, switch between coordination architectures, and develop more robust methods for maintaining swarm cohesion under adverse conditions.

One limitation of the current project was the absence of payload integration, as UAV platforms lacked the necessary mechanisms to carry or deploy substantial payloads. Future work could address this limitation by incorporating payload delivery mechanisms and evaluating their effects on flight performance, stability, and mission execution.

Conclusion

All preliminary simulation results, including test videos and their resulting metrics in a *.csv* file, are provided in the following Dropbox link. Although commands take a considerable amount of time to perform, this can be easily remedied by changing the minimum UAV velocity from 0.4 $\frac{m}{s}$ to higher velocities, with the slower command velocity intended to demonstrate the minutiae of the control schema. A central finding from developing this framework is that preserving software and communication interfaces across simulation and hardware is necessary for systematic comparison, but insufficient to ensure equivalent dynamic behavior. Reliable simulation-to-reality evaluation requires experimental characterization of differences in physical dynamics, sensing uncertainty, state estimation, and computational latency.

This project aimed to develop a standardized UAV benchmarking framework for both individual UAVs and UAV swarms, providing a means to evaluate onboard computational capabilities, communication architectures, and coordination efficiency. MagpieMark establishes a simulation environment for single-UAV benchmarking and a comprehensive set of benchmark tests to evaluate individual and swarm-level performance. Finally, this paper outlines a methodology for constructing a physical benchmark environment analogous to the simulation setup, enabling future experimental validation and quantitative comparisons between simulated and real-world UAV performance.

References

[BNG+16] Michael Burri, Janosch Nikolic, Pascal Gohl, Thomas Schneider, Joern Rehder, Sammy Omari, Markus W. Achtelik, and Roland Siegwart. The EuRoC micro aerial vehicle datasets. *The International Journal of Robotics Research*, 35(10):1157–1163, 2016.

[KBL+23] Elia Kaufmann, Leonard Bauersfeld, Antonio Loquercio, et al. Champion-level drone racing using deep reinforcement learning. *Nature*, 620:982–987, 2023.

[LKR+21] Antonio Loquercio, Elia Kaufmann, René Ranftl, Matthias Müller, Vladlen Koltun, and Davide Scaramuzza. Learning high-speed flight in the wild. *Science Robotics*, 6(59):eabg5810, 2021.

[Mar26] Marvelmind Robotics. Marvelmind Indoor Positioning System. `https://marvelmind.com/`, 2026. Accessed: 2026-10-13.

[MKL+25] Cameron Mehlman, Asterios Kounios, Laurance Lai, Adhyan Prasad, Will Kully, Denis Brown, Ryan Hughes, Shashank Chalamalasetty, Jonathan Distler, Rafael Dilone, Edward Palomino, Keegan Lewick, Matteo Calabrese, Nicolò Boschetti, Mahika Goel, Sean Phillips, and Gregory Falco. The MAGPIE: Satellite autonomy for uncooperative environments. In *Proceedings of the 58th Hawaii International Conference on System Sciences*, pages 7275–7284, 2025.

[Nat25] National Institute of Standards and Technology. Aerial drone tests, April 2025. Last updated April 18, 2025. Accessed October 9, 2026.

[PHSA17] James A. Preiss, Wolfgang Hönig, Gaurav S. Sukhatme, and Nora Ayanian. Crazyswarm: A large nano-quadcopter swarm. In *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3299–3304. IEEE, 2017.

[SNK+21] Yunlong Song, Selim Naji, Elia Kaufmann, Antonio Loquercio, and Davide Scaramuzza. Flightmare: A flexible quadrotor simulator. In *Proceedings of the Conference on Robot Learning*, volume 155 of *Proceedings of Machine Learning Research*, pages 1147–1157. PMLR, 2021.